Django Abstract vs Proxy Models & Signals — Notes

1. Abstract Base Model (Abstract Model) Markdown format

Definition

An abstract base model is used for model inheritance and code reuse.

- It does **not create its own database table**.
- It allows multiple models to reuse common fields and methods.
- Child models inherit the fields and methods defined in the abstract model.
- Use it when multiple models share common fields or behavior.

Example

```
from django.db import models


class CommonInfo(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True


class Student(CommonInfo):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Notes

- `CommonInfo` does not create a database table.
- `Student` creates its own database table.
- The `Student` table includes `created_at`, `updated_at`, `name`, and `age` fields.
- The `abstract = True` option makes `CommonInfo` an abstract base model.
- Abstract models are useful for reusing common fields and methods across multiple models.

Example: Multiple Child Models

```
class Teacher(CommonInfo):
    name = models.CharField(max_length=100)
    subject = models.CharField(max_length=100)
```

```
class Employee(CommonInfo):
    name = models.CharField(max_length=100)
    salary = models.DecimalField(max_digits=10, decimal_places=2)
```

Both `Teacher` and `Employee` inherit `created_at` and `updated_at` from `CommonInfo`, but each has its own database table.

---

2. Proxy Model

Definition

A proxy model allows you to customize the Python-level behavior of an existing Django model without creating a new database table.

- It reuses the original model's database table.
- It does not introduce new database fields.
- It can add custom methods, ordering, or managers.
- It can provide a different Python-level interface to the same database records.

Example

```
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    class Meta:
        ordering = ["name"]


class StudentProxy(Student):
    class Meta:
        proxy = True
        ordering = ["age"]

    def is_adult(self):
        return self.age >= 18
```

Notes

- `StudentProxy` uses the same database table as `Student`.

- The `proxy = True` option defines a proxy model.
- The `is_adult()` method adds custom behavior without adding a database field.
- `StudentProxy` has its own model class and can have different default ordering.
- Use proxy models when you want to customize model behavior without changing the database structure.

Example: Using the Proxy Model

```
student = StudentProxy.objects.get(id=1)

print(student.name)
print(student.is_adult())
```

The proxy model accesses the same student records as the original `Student` model.

---

3. Abstract Model vs Proxy Model

Feature	Abstract Model	Proxy Model
Database table	No table for the abstract parent	Reuses the existing concrete model's table
Field inheritance	Inherits fields into child models	Cannot add new database fields
Methods	Can define reusable methods	Can add or override methods

Purpose	Reuse common fields and behavior	Customize existing model behavior
Database structure	Child models have their own tables	No additional table
`Meta` option	`abstract = True`	`proxy = True`
Typical use case	Shared fields across multiple models	Custom ordering, methods, or managers

Key Differences

Abstract Model

- Used for structural reuse.
- Child models inherit fields and methods.
- Each concrete child model gets its own database table.

Proxy Model

- Used for behavioral customization.
- Reuses an existing concrete model's table.
- Cannot add new database fields.

Remember

- **Abstract Model = Reuse fields and methods.**
- **Proxy Model = Reuse the existing table and customize behavior.**

Django Signals

1. What Are Django Signals?

Django Signals allow certain senders to notify registered receivers when specific events occur.

In simple terms, a receiver function can execute automatically when a particular event happens.

Example: Automatically creating a Profile when a new User is created.

Main Components

- **Sender:** The model or process that triggers an event.
- **Signal:** The notification sent when an event occurs.
- **Receiver:** The function that responds to the signal.
- **Dispatcher:** The mechanism that connects senders and receivers.

Basic Workflow

```
Event Occurs
     |
     v
Signal Is Sent
     |
     v
Registered Receiver Executes
```

2. Python Event-Driven Concept

Django Signals use a concept similar to callback functions.

```
def send_notification(username):
    print(f"Welcome, {username}!")


def create_user(username, callback):
    print(f"User {username} created.")
    callback(username)


create_user("Faruk", send_notification)
```

Output:

```
User Faruk created.
Welcome, Faruk!
```

Here, `send_notification` is a callback function passed to `create_user`.

3. Important Built-in Django Signals

Signal	Purpose
`pre_save`	Sent before a model instance is saved
`post_save`	Sent after a model instance is saved
`pre_delete`	Sent before a model instance is deleted
`post_delete`	Sent after a model instance is deleted
`m2m_changed`	Sent when a many-to-many relationship changes
`pre_init`	Sent before a

	model instance is initialized
`post_init`	Sent after a model instance is initialized
`request_started`	Sent when Django begins handling a request
`request_finished`	Sent when Django finishes handling a request

4. `pre_save` vs `post_save`

`pre_save`

The `pre_save` signal is sent before Django saves a model instance.

```
from django.db.models.signals import pre_save
from django.dispatch import receiver
from django.contrib.auth.models import User


@receiver(pre_save, sender=User)
def before_user_save(sender, instance, **kwargs):
    print("User is about to be saved.")
```

`post_save`

The `post_save` signal is sent after Django saves a model instance.

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth.models import User


@receiver(post_save, sender=User)
def after_user_save(sender, instance, created, **kwargs):
    if created:
        print("A new user was created.")
    else:
        print("An existing user was updated.")
```

Important: `post_save` does not guarantee that the surrounding database transaction has committed.

5. Django Signals Project Setup

Assume that the project is named `config` and the application is named `accounts`.

Project Structure

```
myproject/
├── config/
│   ├── settings.py
│   └── urls.py
├── accounts/
│   ├── apps.py
│   ├── models.py
│   ├── signals.py
│   ├── views.py
│   └── tests.py
└── manage.py
```

Step 1: Create the App

```
python manage.py startapp accounts
```

Step 2: Register the App

In `settings.py`:

```
INSTALLED_APPS = [
    # Other apps...
```

```
    "accounts.apps.AccountsConfig",
]
```

Step 3: Create `signals.py`

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth.models import User


@receiver(post_save, sender=User)
def user_saved(sender, instance, created, **kwargs):
    if created:
        print(f"New user created: {instance.username}")
```

Step 4: Register Signals in `apps.py`

```
from django.apps import AppConfig


class AccountsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "accounts"

    def ready(self):
        import accounts.signals
```

Why Use `ready()`?

Django calls the `ready()` method during application initialization. Importing the signals module here ensures that its receiver functions are registered.

Creating `signals.py` alone is not enough; the module must be imported.

Best practice: Avoid database queries inside `ready()`. If appropriate, use `dispatch_uid` to prevent duplicate receiver registration.

6. Automatically Create a User Profile

This is one of the most common real-world uses of Django Signals.

Step 1: Create the Profile Model

In `accounts/models.py`:

```
from django.db import models
from django.contrib.auth.models import User
```

```
class Profile(models.Model):
    user = models.OneToOneField(
        User,
        on_delete=models.CASCADE,
        related_name="profile",
    )
    bio = models.TextField(blank=True)
    phone = models.CharField(max_length=15, blank=True)

    def __str__(self):
        return self.user.username
```

The `OneToOneField` ensures that each User can have at most one associated Profile.

Step 2: Create the Signal

In `accounts/signals.py`:

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth.models import User

from .models import Profile


@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

Step 3: Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 4: Test in the Django Shell

```
python manage.py shell
```

```
from django.contrib.auth.models import User
from accounts.models import Profile

user = User.objects.create_user(
    username="faruk",
    password="strong-test-password",
```

```
)

profile = Profile.objects.get(user=user)

print(profile.user.username)
```

Output:

```
faruk
```

Why Check `created`?

- `created=True`: A new database record was created.
- `created=False`: An existing record was saved.

Checking `created` prevents the receiver from attempting to create another Profile every time an existing User is updated.

Alternative: `get_or_create()`

```
@receiver(post_save, sender=User)
def ensure_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.get_or_create(user=instance)
```

This helps avoid creating a duplicate Profile if one already exists.

7. `pre_save` Example: Normalize a Product Name

In `shop/models.py`:

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=200)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
```

In `shop/signals.py`:

```
from django.db.models.signals import pre_save
from django.dispatch import receiver

from .models import Product
```

```
@receiver(pre_save, sender=Product)
def normalize_product_name(sender, instance, **kwargs):
    instance.name = instance.name.strip()
```

Usage:

```
product = Product(
    name="  Wireless Mouse  ",
    price=500,
)

product.save()

print(product.name)
```

Output:

```
Wireless Mouse
```

The receiver removes leading and trailing whitespace before the Product is saved.

8. `pre_delete` and `post_delete`

These signals allow you to execute code before or after a model instance is deleted.

```
from django.db.models.signals import pre_delete, post_delete
from django.dispatch import receiver

from .models import Product


@receiver(pre_delete, sender=Product)
def before_product_delete(sender, instance, **kwargs):
    print(f"Deleting product: {instance.name}")


@receiver(post_delete, sender=Product)
def after_product_delete(sender, instance, **kwargs):
    print(f"Deleted product: {instance.name}")
```

- `pre_delete`: Sent before the delete operation.
- `post_delete`: Sent after the delete operation.

After deletion, the database row no longer exists, but the in-memory `instance` can still provide its field values.

9. `m2m_changed` Signal

The `m2m_changed` signal is sent when a many-to-many relationship changes through Django's relationship manager.

Example Models

```
from django.db import models


class Category(models.Model):
    name = models.CharField(max_length=100)


class Product(models.Model):
    name = models.CharField(max_length=200)
    categories = models.ManyToManyField(
        Category,
        related_name="products",
        blank=True,
    )
```

Signal Receiver

```
from django.db.models.signals import m2m_changed
from django.dispatch import receiver

from .models import Product


@receiver(
    m2m_changed,
    sender=Product.categories.through,
)
def product_categories_changed(
    sender,
    instance,
    action,
    reverse,
    model,
    pk_set,
    **kwargs,
):
    if action == "post_add":
```

```
        print("Categories added.")

    elif action == "post_remove":
        print("Categories removed.")

    elif action == "post_clear":
        print("All categories cleared.")
```

Important Actions

Action	Meaning
`pre_add`	Before relationships are added
`post_add`	After relationships are added
`pre_remove`	Before relationships are removed
`post_remove`	After relationships are removed
`pre_clear`	Before all relationships are cleared
`post_clear`	After all relationshi

	ps are cleared

The value of `pk_set` depends on the action and relationship direction. It may be `None` for clear operations.

10. Custom Signals

Custom signals allow you to define application-specific events.

Step 1: Define a Signal

In `orders/events.py`:

```
import django.dispatch

order_placed = django.dispatch.Signal()
```

Step 2: Create a Receiver

In `orders/receivers.py`:

```
from django.dispatch import receiver
from .events import order_placed


@receiver(order_placed)
def handle_order_placed(sender, order, **kwargs):
    print(f"Order placed: {order.id}")
```

Import this receiver module during application initialization to register the receiver.

Step 3: Send the Signal

In `orders/services.py`:

```
from .events import order_placed


def notify_order_placed(order):
    order_placed.send(
        sender=order.__class__,
        order=order,
    )
```

Usage:

```
order = ...  # An existing Order instance
notify_order_placed(order)
```

The receiver receives `order` as a keyword argument.

When to Use Custom Signals

- Notifying independent components about an event.
- Providing extension points in reusable applications.
- Connecting multiple independent receivers to one event.

For workflows requiring strict execution order, explicit service functions are often easier to maintain.

11. Django Signals vs Model `save()` Override

Feature	Signals	`save()` Override
Purpose	Event handling	Customize model save behavior
Multiple independent handlers	Supported	Not directly
Code location	Can be in a separate module	Usually in the model class
Execution flow	Requires locating the receiver	Visible in the model class

Best use	Independent side effects	Model-specific save rules

Example: `save()` Override

```
class Product(models.Model):
    name = models.CharField(max_length=200)

    def save(self, *args, **kwargs):
        self.name = self.name.strip()
        super().save(*args, **kwargs)
```

For a model's own normalization rules, a `save()` override or validation logic may be clearer than a signal.

12. `transaction.on_commit()`

Use `transaction.on_commit()` to execute a callback after the active database transaction successfully commits.

Example: Send an Email After an Order Is Created

```
from django.db import transaction
from django.db.models.signals import post_save
from django.dispatch import receiver

from .models import Order


def send_order_email(order_id):
    print(f"Sending email for order {order_id}")


@receiver(post_save, sender=Order)
def order_created(sender, instance, created, **kwargs):
    if created:
        transaction.on_commit(
            lambda: send_order_email(instance.pk)
        )
```

Why Is This Important?

- The callback runs after a successful commit.

- If the transaction rolls back, the callback is discarded.
- If there is no active transaction, the callback normally executes immediately.

Common uses include:

- Sending emails.
- Queuing background tasks.
- Invalidating caches.
- Notifying external services.

For critical business events, also consider failure handling, retries, and the transactional outbox pattern.

13. Django Signals with Celery

Heavy tasks can be delegated to a background worker using Celery.

In `products/tasks.py`:

```
from celery import shared_task


@shared_task
def process_product_image(product_id):
    print(f"Processing product image: {product_id}")
```

In `products/signals.py`:

```
from django.db import transaction
from django.db.models.signals import post_save
from django.dispatch import receiver

from .models import Product
from .tasks import process_product_image


@receiver(post_save, sender=Product)
def product_saved(sender, instance, created, **kwargs):
    if created:
        product_id = instance.pk

        transaction.on_commit(
            lambda: process_product_image.delay(product_id)
        )
```

Workflow

1. Django Signal detects the event.
2. `transaction.on_commit()` schedules the callback after a successful commit.
3. Celery executes the task in a background worker.

Note: This pattern does not guarantee exactly-once task execution. Production systems should consider retries, idempotency, and task-delivery failures.

14. Django REST Framework (DRF) and Signals

Django model signals also work when a model instance is created through Django REST Framework, provided the operation uses the normal model save lifecycle.

```
from rest_framework import serializers
from django.contrib.auth.models import User


class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ["id", "username", "password"]
        extra_kwargs = {
            "password": {"write_only": True}
        }

    def create(self, validated_data):
        return User.objects.create_user(**validated_data)
```

When `create_user()` creates a User, the registered `post_save` receiver also runs. However, operations such as `bulk_create()` and `QuerySet.update()` generally bypass the normal per-instance save signals.

15. Important Signal Parameters

Example receiver:

```
@receiver(post_save, sender=Product)
def product_saved(
    sender,
    instance,
    created,
    raw,
    using,
    update_fields,
```

```
    **kwargs,
):
    pass
```

Parameter	Meaning
`sender`	The model class that sent the signal
`instance`	The affected model instance
`created`	Whether a new record was created
`raw`	Whether the save is occurring in raw mode
`using`	The database alias used
`update_fields`	The set of fields

	being updated, or `None`
`**kwargs`	Additional keyword arguments

Not every signal provides the same parameters. Write receiver signatures according to the signal being used.

16. Common Mistakes and Best Practices

Mistake 1: Not Importing the Signals Module

Creating `signals.py` does not automatically register its receivers.

Solution: Import the signals module from `AppConfig.ready()`.

Mistake 2: Infinite Recursion

```
@receiver(post_save, sender=Product)
def bad_signal(sender, instance, **kwargs):
    instance.save()
```

Calling `instance.save()` inside the same model's `post_save` receiver can trigger the receiver again.

Mistake 3: Putting Too Much Business Logic in Signals

Avoid placing long workflows directly inside signal receivers.

Better approach: Use service functions or Celery tasks where appropriate.

Best Practices

- Keep receivers small and focused.
- Use the `created` parameter correctly.
- Use `transaction.on_commit()` for transaction-sensitive actions.
- Use `dispatch_uid` when needed to prevent duplicate registration.
- Write automated tests.
- Do not assume that bulk operations trigger save signals.
- Handle `raw=True` carefully during fixture loading.
- Avoid database queries inside `AppConfig.ready()`.

Important Limitations

- `bulk_create()` generally does not trigger `pre_save` or `post_save`.
- `QuerySet.update()` does not call each model's `save()` method or its save signals.
- `post_save` does not guarantee transaction commit.
- Receiver exceptions can cause the calling operation to fail.
- During raw fixture loading, avoid modifying other database objects unless the operation is specifically designed for it.

17. Testing Django Signals

In `accounts/tests.py`:

```
from django.test import TestCase
from django.contrib.auth.models import User

from .models import Profile


class UserProfileSignalTest(TestCase):

    def test_profile_created_with_user(self):
        user = User.objects.create_user(
            username="faruk",
            password="test-password",
        )

        self.assertTrue(
            Profile.objects.filter(user=user).exists()
        )
```

Run the tests:

```
python manage.py test accounts
```

Additional Tests

- Verify that updating a User does not create a duplicate Profile.
- Verify that the receiver is registered.
- Verify that transaction rollback discards an `on_commit()` callback.
- Test the expected behavior when a receiver raises an exception.

18. Django Signals Interview Questions and Answers

Q1. What are Django Signals?

Django Signals allow senders to notify registered receivers when specific events occur.

Q2. What is the difference between `pre_save` and `post_save`?

`pre_save` is sent before the save operation, while `post_save` is sent after it.

Q3. What does `created=True` mean?

It indicates that a new database record was created.

Q4. Why use `AppConfig.ready()`?

It provides a standard place to import the signals module during application initialization and register receivers.

Q5. Can signals cause recursion?

Yes. Calling `save()` inside the same model's `post_save` receiver can trigger the receiver repeatedly.

Q6. Do `bulk_create()` and `QuerySet.update()` trigger save signals?

Generally, no.

Q7. What is `transaction.on_commit()`?

It registers a callback that executes after the active database transaction successfully commits.

Q8. Can multiple receivers listen to the same signal?

Yes. Multiple receivers can connect to the same signal.

Q9. What is a custom signal?

A signal defined by an application for its own events using `django.dispatch.Signal`.

Q10. When should you avoid Django Signals?

Avoid them when an explicit service call is clearer or when execution order, error handling, and return values need explicit control.

19. Practice Tasks

Beginner

1. Automatically create a Profile when a User is created.
2. Create an AuditLog record when a Product is created or deleted.
3. Normalize Product names using `pre_save`.

Intermediate

4. Track Category changes using `m2m_changed`.
5. Queue an Order confirmation task using `transaction.on_commit()`.

6. Write tests to verify duplicate Profile prevention.

Advanced

7. Integrate Django Signals with Celery.
8. Implement appropriate failure handling and retries.
9. Learn and implement the transactional outbox pattern.

20. Quick Revision Sheet

Topic	Remember
`pre_save`	Before saving
`post_save`	After saving
`pre_delete`	Before deleting
`post_delete`	After deleting
`m2m_changed`	Many-to-many relationship changes
`created`	Checks whether a new record was created

`AppConfig.ready()`	Register signal receivers
`transaction.on_commit()`	Execute a callback after commit
Celery	Background task execution
`dispatch_uid`	Helps prevent duplicate receiver registration

Recommended Learning Order

1. Built-in Signals
2. `post_save` and the Profile example
3. Signal registration with `AppConfig`
4. `pre_save` and delete signals
5. Custom Signals
6. Transactions and `on_commit()`
7. Celery integration
8. Testing and interview preparation

Final Takeaway: Django Signals simplify event-driven programming. For production-quality backend development, it is equally important to understand model lifecycle behavior, database transactions, error handling, and automated testing.